
AdaptMath

Adaptivní vzdělávání pro matematické disciplíny

Technická dokumentace

Task Checker — nástroj pro plnění a kontrolu úloh

Projekt: TQ23000206 (TA ČR)
Hlavní řešitel: Mgr. Jana Medková, Ph.D.
Pracoviště: Fakulta informatiky a managementu, Univerzita Hradec Králové
Doba řešení: 09/2025 – 02/2029
Autor dokumentace: Mgr. Dominik Palla
Verze: 1.0
Datum: 27. května 2026

Obsah

1	Úvod	2
1.1	Cíl projektu	2
1.2	Co řeší tato dokumentace	2
1.3	Tým	2
2	Architektura	2
2.1	Technologický stack	2
2.2	Struktura repozitáře	3
2.3	Tok dat	3
3	Datový model	3
3.1	Tabulka <code>math_tasks</code>	3
3.2	Konvence <code>task_id</code>	3
3.3	Struktura <code>results</code>	4
3.4	Typy výsledků	4
4	Vektor znalostí a kategorie	4
4.1	Filozofie	4
4.2	<code>KNOWLEDGE_WEIGHTS</code> (67 vah)	4
4.3	<code>TASK_CATEGORIES</code> (51 kategorií)	5
5	Webové UI: Task Checker	5
5.1	Routes	5
5.2	Editor úlohy (<code>/tasks/<id></code>)	5
5.3	Tipy pro vyhodnocení MathLive Compute Engine	6
6	Datový obsah	6
6.1	Zdroj úloh	6
6.2	Pokrytí	7
6.3	Distribuce typů výsledků	7
7	Spuštění lokálně	7
7.1	Závislosti	7
7.2	První spuštění	7
7.3	Opětovné nasazení po změně schématu	8
8	Co dál (roadmap)	8
8.1	Krátkodobé	8
8.2	Střednědobé	8
8.3	Dlouhodobé	8
9	Reference	8

1. Úvod

1.1. Cíl projektu

AdaptMath je projekt aplikovaného výzkumu TA ČR (TQ23000206, program SIGMA), jehož cílem je navrhnout a pilotně ověřit nové přístupy k adaptivnímu vzdělávání v matematických oborech na vysokých školách. Klíčovým prvkem je vytvoření *adaptivního engine*, který na základě profilu studenta, klasifikace úloh a jeho odpovědí dynamicky přizpůsobuje výběr další úlohy a tvorbu personalizovaného kurzu.

Plánované výstupy projektu:

- **TQ23000206-V2** — adaptivní engine pro matematické kurzy (software v Pythonu),
- **TQ23000206-V3** — adaptivní kurzy pro výuku matematiky (integrace v Moodle).

1.2. Co řeší tato dokumentace

Dokumentace popisuje **Task Checker** — pomocnou webovou aplikaci určenou pro fázi *plnění a kontroly obsahu*. Aplikace umožňuje výzkumnému týmu:

- importovat matematické úlohy ze zdrojových `.tex` skript do databáze,
- vizuálně procházet a ověřovat zadání i jejich očekávané výsledky,
- editovat všechny atributy úlohy (zadání, výsledky, kategorie, vektor vah, IRT parametry),
- testovat odpovědi pomocí **MathLive Compute Engine** stejným způsobem, jakým je bude vyhodnocovat budoucí studentský engine.

Adaptivní engine (IRT, BKT, personalizace) zatím není součástí — bude přidán v následující fázi projektu.

1.3. Tým

Člen	Role	Odpovědnost
dr. Jana Medková	hlavní řešitel	vize datového modelu, vedení projektu
dr. Jiří Haviger	IRT engine	<code>py-irt</code> , kalibrace obtížnosti
dr. Eva Ševčíková	BKT modely	<code>py-bkt</code> , učící parametry α, β
dr. Petr Bauer	obsah	spoluautor skriptu <i>Základy matematiky 1</i>
dr. Štefan Šteflovíč	člen týmu	
dr. Dominik Palla	vývoj	frontend, integrace, LLM funkce

2. Architektura

2.1. Technologický stack

Vrstva	Technologie
Backend	Python 3.10+, Flask 3.1, SQLAlchemy 2.x, psycopg2
Databáze	PostgreSQL 15 (v Dockeru)
Frontend (render)	KaTeX 0.16 — sazba matematiky
Frontend (vstup)	MathLive (<code><math-field></code> web component)
Vyhodnocení odpovědí	MathLive Compute Engine (LaTeX → MathJSON)
Šablony	Jinja2 (server-side render)
Kontejnerizace	Docker Compose
Frontend libraries	CDN (jsdelivr, unpkg) — žádný build krok

2.2. Struktura repozitáře

```

adapt_math/
  app.py                # Flask aplikace (routes, API)
  database.py           # init_db (SQLAlchemy engine + session)
  model.py              # ORM model MathTask
  seed_db.py            # drop & recreate + napln. uloh
  docker-compose.yml    # PostgreSQL kontejner
  tasks/
    __init__.py         # def. uloh, jeden soubor = jedno cviceni
    knowledge_weights.py # agreguje ALL_TASKS
    cv01.py ... cv13.py # ulohy ze skripta "Zaklady matematiky 1"
  templates/
    tasks_list.html     # seznam vseh uloh (filtr)
    task_checker.html    # editor jedne ulohy + sandbox

```

2.3. Tok dat

1. Zdrojové .tex skripty (cvičení 1–13) jsou ručně přepsány do strukturovaných Python slovníků v `tasks/cvNN.py` (LLM-asistovaná extrakce, viz sekce 6).
2. `seed_db.py` smaže a znovu vytvoří tabulku `math_tasks` a vloží do ní všech 415 úloh přes SQLAlchemy.
3. `app.py` (Flask) servíruje webové UI; přes endpoint `POST /api/tasks/<id>` expert ukládá změny.
4. Frontend MathLive Compute Engine vyhodnocuje studentské odpovědi symbolicky / numericky, bez backend kola.

3. Datový model

3.1. Tabulka `math_tasks`

Definováno v `model.py` (SQLAlchemy ORM). Aktuálně jediná tabulka.

Sloupec	Typ	Popis
<code>task_id</code>	String (PK)	Identifikátor stylu <code>cvNN_i</code> (např. <code>cv04_5</code>).
<code>content_latex</code>	String	Zadání úlohy v LaTeXu; KaTeX render. Pure-math (bez \$) se v UI automaticky obalí do <code>\$\$...\$\$</code> .
<code>results</code>	JSON	Pole pojmenovaných výsledků (viz 3.3).
<code>category</code>	String	Primární téma úlohy (jeden řetězec z 51 <code>TASK_CATEGORIES</code>).
<code>knowledge_vector</code>	JSON	Slovník <code>{name: int 0-100}</code> ; podíl skill-komponent pro distribuci přírůstku znalosti. Ukládají se jen non-zero hodnoty.
<code>cognitive_load</code>	String	Klasifikace A–F (typ / praktičnost úlohy).
<code>graph_vector</code>	JSON	<i>Deprecated</i> . Starší tag-vektor (list stringů). Zachováno pro kompatibilitu, v UI se needituje.
<code>irt_difficulty</code>	Float	IRT obtížnost (± 3). Doladí Haviger.
<code>irt_discrimination</code>	Float	IRT diskriminace ($\pm 2,5$).

3.2. Konvence `task_id`

Formát `cvNN_i`, kde:

- NN je dvouciferné číslo cvičení (01–13),
- i je pořadové číslo úlohy v rámci cvičení (1, 2, ...).

Příklady: cv01_4 (4. úloha cv01), cv12_58 (poslední úloha cv12).

3.3. Struktura results

Pole **results** drží **kolekci pojmenovaných výsledků**. Každý prvek pole reprezentuje jeden očekávaný vstup od studenta. Úloha tak může mít více polí (např. $D(f) = \dots$, *intervaly růstu*, *lokální extrém v $x = \dots$*).

Schema jednoho prvku:

```
{
  "key": "<identifikátor>",          # interní, pro logging
  "label_latex": "<LaTeX>",          # prefix před vstupem, např. "D(f) = "
  "type": "<typ>",                   # decimal | mathlive | multiple_choice |
    open_text
  "expected": <hodnota>,             # podle typu (viz níže)
  "tolerance": <float>,              # pouze pro type=decimal
  "options": [...],                 # pouze pro type=multiple_choice
}
```

3.4. Typy výsledků

Typ	expected	Vyhodnocení
decimal	float	MathLive vstup parsovaný přes Compute Engine <code>.N().valueOf()</code> ; porovnání numericky s <code>tolerance</code> .
mathlive	string (LaTeX)	Compute Engine 3-fázová strategie: <code>canonical.isSame() → simplify().isSame() → .N()</code> numericky.
multiple_choice	klíč správné možnosti	Radio buttony; <code>options[]</code> je pole <code>{key, label_latex}</code> .
open_text	vzorové řešení (string)	Žádné auto-vyhodnocení; doporučuje LLM (Gemini) nebo expertní revizi.

4. Vektor znalostí a kategorie

4.1. Filozofie

Každá úloha trénuje různé *skill-komponenty* v různém poměru. **knowledge_vector** je slovník přiřazující každé komponentě procentuální váhu (0–100). Při řešení úlohy se přírůstek (úspěch) nebo úbytek (nesplnění) studentovy znalosti rozdělí mezi komponenty v poměru těchto vah.

Paralelní vektor existuje i ve studentově profilu — tam ale hodnoty $\in [0, 1]$ (úroveň znalosti v dané komponentě, IRT/BKT).

4.2. KNOWLEDGE_WEIGHTS (67 vah)

Definováno v `tasks/knowledge_weights.py`. Rozděleno do dvou kategorií:

(A) Algebraické a aritmetické skill-komponenty (16 vah). *lineární, kvadratická, mocnná, log, exponenciální, goniometrická, Absolutní hodnota, lomená lineární, zlomky, Aplikační, vzorec, vytýkání–krácení, Roznásobení závorky — distributivní zákon, výpočet rovnic, výpočet nerovnic, Derivování.*

(B) Kategorie úloh (51 vah). Začínají od *Algebraické výrazy* a pokračují přes *Elementární funkce, Rovnice, Nerovnice, Soustavy rovnic, Funkce, Definiční obor, Aritmetika funkcí, Skládání, Sudá/lichá, Inverzní funkce, Tečna, Asymptota, Monotonie, Absolutní extrémy, Konverznost, Spojitost, Bolzanova věta, Limita* (a její podtypy VOAL, LVL, S odmocninou, Vytknutí nejvyšší mocniny, Jednostranné, Typová, L'Hospital), *Derivace* (a její podtypy Sčítání, Součin, Podíl, Složená funkce, Diferenciál, Taylor, vyšších řádů, Průběh funkce), *Primitivní funkce* (Sčítání, Per partes, 1./2. věta o sub, Parciální zlomky), *Určitý integrál* (UI variants), *Nevlastní integrál, Obsah plochy*.

Duplicitní názvy. *Sčítání* se přirozeně objevuje jako podtyp *Derivace* i *Primitivní funkce*. Rozlišujeme prefixem: "Derivace -- Sčítání" vs "Prim. funkce -- Sčítání". Varianty pro určitý integrál nesou prefix "UI - " (např. *UI - Per partes*).

4.3. TASK_CATEGORIES (51 kategorií)

Subset KNOWLEDGE_WEIGHTS začínající od indexu 16 (*Algebraické výrazy*). Tato množina se nabízí v UI editoru jako `<select>` pro pole *Kategorie úlohy*.

V další fázi (zatím **není implementováno**) bude každá kategorie mít defaultní distribuci vah, která se automaticky aplikuje na `knowledge_vector` při výběru kategorie, a expert ji pak doladí na úrovni jednotlivé úlohy.

5. Webové UI: Task Checker

5.1. Routes

Metoda	Cesta	Účel
GET	/	Redirect na první úlohu v abecedním pořadí.
GET	/tasks	Tabulka všech úloh (filtr podle tématu, fulltext).
GET	/tasks/<id>	Detail úlohy — editor + sandbox.
POST	/api/tasks/<id>	Uložit změny (JSON body s editable poli).
DELETE	/api/tasks/<id>	Smazat úlohu.

5.2. Editor úlohy (/tasks/<id>)

UI je rozděleno do dvou sloupců.

Levý sloupec (vizualizace + test).

- Živý KaTeX náhled zadání (re-renderuje při každém keystroke).
- Sandbox „Vyzkoušej řešení“ — MathLive vstupní pole; tlačítko *Otestovat* spustí vyhodnocení proti aktuálnímu (i neuloženému) **expected**.
- Sekce „Vzorové odpovědi“ — vyrenderované očekávané hodnoty.

Pravý sloupec (editor).

- Task ID, kognitivní zátěž (select A–F), kategorie úlohy (select 51 položek).
- Zadání v LaTeXu (textarea, full-width).
- Editor výsledků — karta pro každý prvek pole **results**; lze přidat/odebrat; pro každý se mění **key**, **label_latex**, typ (select), a **expected** (specifické pro typ — MathLive pole, číselný input, MC options, textarea).

Pod editorem (full-width).

- **Vektor znalostí** — plochý seznam 67 vah ve 2 sloupcích; každý řádek má label, range slider (0–100, krok 1), spojený number input, znak %.
- Filtr textový („lim“, „derivace“) + checkbox *jen nenulové*.
- Sumarizace v hlavičce sekce: aktuální součet všech vah, počet nenulových z 67, tlačítko *Vynulovat vše*.
- Slider $\leftrightarrow$ number input jsou dvousměrně synchronizované; aktivní řádky (váha > 0) jsou zvýrazněné a uložené jako jediné v JSON (chybějící klíč = 0).

Sticky save bar (dole).

- *Uložit změny* — POST `/api/tasks/<id>` s celým stavem;
- *Vrátit* — obnoví poslední uložený stav;
- *Smazat úlohu* — DELETE endpoint.

Klávesové zkratky. $\leftarrow$ a $\rightarrow$ pro navigaci mezi úlohami, Ctrl+S (resp. Cmd+S) pro uložení.

5.3. Tipy pro vyhodnocení MathLive Compute Engine

- **mathlive** odpovědi se ukládají jako čistý LaTeX. Compute Engine je tolerantní k variantám zápisu zlomku, znamének, `\infty` vs `+\infty`, atd.
- Pro intervaly preferujeme standardní zápis: (a, b) otevřený, $[a, b]$ uzavřený, `\cup` pro sjednocení.
- Tam, kde by Compute Engine selhal (cyklické sjednocení po $k \in \mathbb{Z}$, „limita neexistuje“ jako popis), používáme **multiple_choice** se třemi variantami (1 správná + 2 typické chyby).

6. Datový obsah**6.1. Zdroj úloh**

Skriptu *Základy matematiky 1* (P. Pražák, P. Bauer; KIKM FIM UHK). Zdrojové `.tex` soubory cvičení 1–13 byly ručně (LLM-asistovaně) interpretovány a převedeny do strukturovaného formátu v `tasks/cvNN.py`.

6.2. Pokrytí

Cvičení	Téma	Úloh
cv01	Funkce, definiční obor, kvadratická funkce	37
cv02	Skládání, prostá, inverzní funkce	29
cv03	Monotonie, parita, polynomy, Hornerovo schéma	42
cv04	Limita — aritmetika limit	18
cv05	Limita — věta o složené funkci, typové limity	44
cv06	Spojitosť funkce	23
cv07	Derivace (součet, součin, podíl, řetězkové pravidlo)	47
cv08	Tečna, normála, absolutní extrém, L'Hospital	36
cv09	1. derivace, lokální extrém, optimalizace	26
cv10	2. derivace (konvexita, inflexe), asymptoty	18
cv11	Diferenciál, Taylorův a Maclaurinův polynom	27
cv12	Primitivní funkce (linearita, per-partes, substituce)	58
cv13	Integrace racionálních funkcí	10
Celkem		415

6.3. Distribuce typů výsledků

Typ	Počet
mathlive	319
multiple_choice	115
decimal	83
open_text	0 *

*Původní negace výroků byly převedeny na multiple_choice se 3 možnostmi (správná + 2 typické chyby studentů).

7. Spuštění lokálně

7.1. Závislosti

- Python 3.10+
- Docker (pro PostgreSQL)
- Python balíčky — viz .venv: flask, sqlalchemy, psycopg2-binary, google-generativeai (volitelné, pro LLM nápovědu)

7.2. První spuštění

```
# 1) klon repozitáře
git clone https://github.com/dominikpalla/adapt_math.git
cd adapt_math

# 2) Python venv + dependencies
python3 -m venv .venv
source .venv/bin/activate
pip install flask sqlalchemy psycopg2-binary google-generativeai python-dotenv

# 3) Spustit PostgreSQL kontejner
docker compose up -d
```



```
# 4) Naplnit DB (drop & recreate + insert 415 úloh)
python seed_db.py

# 5) Spustit Flask
python app.py
```

Aplikace běží na <http://127.0.0.1:5000>.

7.3. Opětovné nasazení po změně schématu

Jelikož v aktuální fázi nepoužíváme migrace (Alembic), po každé změně sloupců v `model.py` stačí spustit znovu `python seed_db.py` — skript provede `DROP TABLE ... CASCADE` a `CREATE TABLE` dle aktuálního ORM modelu, načte úlohy vloží znovu.

Pozor: drop & recreate ztrácí veškerá data nasazená přes UI (ručně upravené kategorie, vektory znalostí). Pro produkci je nutné zavést migrace.

8. Co dál (roadmap)

8.1. Krátkodobé

1. **Distribuce vah dle kategorie.** Definovat slovník `CATEGORY_DEFAULT_VECTOR`, který pro každou ze 51 kategorií předvyplní výchozí distribuci 67 vah. Při výběru kategorie v UI se předvyplní `knowledge_vector`, expert pak ladí.
2. **LLM vyhodnocení open_text.** Aktuálně máme všechny negace převedeny na `multiple_choice`; pro budoucí volné úlohy zavést endpoint `POST /api/eval-open-text` volající Gemini.
3. **Alembic migrace.** Nahradit drop & recreate řízenými migracemi.

8.2. Střednědobé

1. **IRT engine** (`py-irt`). Pilotní tréninková fáze pro kalibraci obtížnosti a diskriminace. Vlastní route `/student/<id>/next-task`, která vybere úlohu podle aktuálního θ studenta.
2. **BKT engine** (`py-bkt`). Po každé interakci aktualizuje studentův vektor znalostí na základě `knowledge_vector` úlohy a parametrů α , β .
3. **Studentský profil.** Tabulka `students` s preferencemi (styl učení, motivace, math anxiety) a vektorem znalostí (67 hodnot v $[0, 1]$).
4. **Logování interakcí** pro výzkum (čas řešení, jistota, AI nápověda).

8.3. Dlouhodobé

1. **Integrace s Moodle.** Engine jako externí Moodle plugin / LTI tool. Výstup TQ23000206-V3.
2. **Generování úloh LLM.** Tvorba variant na téma „kategorie + obtížnost“.
3. **Dashboard pro experty.** Vizualizace pokroku jednotlivých studentů, analýza odchylek od průměru.

9. Reference

- Repozitář projektu: https://github.com/dominikpalla/adapt_math
- Skripta *Základy matematiky 1*, P. Pražák, P. Bauer, FIM UHK
- MathLive: <https://github.com/arnog/mathlive>
- Compute Engine: <https://cortexjs.io/compute-engine/>
- KaTeX: <https://katex.org/>

- py-irt: <https://github.com/nd-ball/py-irt>
- py-bkt: <https://github.com/CAHLR/pyBKT>

Tato dokumentace byla vygenerována ve fázi plnění databáze úloh (Task Checker, verze 1.0). Bude rozšířena po implementaci IRT/BKT engine a integrace s Moodle.